

Written by: Dr. Farkhana Muchtar

# LAB 3: LINUX FILE SYSTEM AND BASH SCRIPTING

## Introduction of File System and Shell Script

### 1 Overview

To learn and understand the Linux File System and basics of Bash scripting, with a focus on file permissions, different types of file paths, and the creation of symbolic and hard links.

### 2 File System and File Permissions

#### 2.1 Exercise 1: Understanding Directories and Basic Navigation

##### 1. Display Your Current Location

- Use the `pwd` command to show your current working directory (where you are now in the filesystem).

**Command:**

```
pwd
```

##### 2. Create a Directory in Your Home Folder

- Your home directory is where your personal files and folders typically reside.
- Create a new directory called `LabExercise` inside your home directory using `~` to represent your home directory.

**Command:**

```
mkdir ~/LabExercise
```

##### 3. Change to the New Directory

- Move into the `LabExercise` directory. This time we use the absolute path (`~/LabExercise`).

**Command:**

```
cd ~/LabExercise
```

## SECR2043 - OPERATING SYSTEMS (Lab Exercise)

Written by: Dr. Farkhana Muchtar

### 4. Verify Your Current Directory

- Check that you are now inside the LabExercise directory.

#### **Command:**

```
pwd
```

### 5. Create a Subdirectory

- Inside LabExercise, create a new directory named SubDir1.

#### **Command:**

```
mkdir SubDir1
```

### 6. Navigate into SubDir1

- Move into the SubDir1 directory using a relative path. Since we are currently in LabExercise, SubDir1 is directly inside it.

#### **Command:**

```
cd SubDir1
```

### 7. Verify Your Current Directory

- Check your location again to ensure you are now in SubDir1.

#### **Command:**

```
pwd
```

### 8. Create Another Subdirectory

- Make a new directory called SubDir2 inside SubDir1.

#### **Command:**

```
mkdir SubDir2
```

# SECR2043 - OPERATING SYSTEMS (Lab Exercise)

Written by: Dr. Farkhana Muchtar

## 2.2 Exercise 2: Using Absolute Paths

An **absolute path** is the complete address of a file or directory from the root (/). It always starts with / and does not depend on where you currently are.

1. Return to Your Home Directory

- Typing `cd` without any arguments takes you back to your home directory.

**Command:**

```
cd
```

2. Navigate with an Absolute Path

- Change directory to `LabExercise` using its absolute path. Replace `<your-username>` with your actual username.

**Command:**

```
cd /home/<your-username>/LabExercise
```

3. Verify Your Position

- Confirm that you are now in `LabExercise`.

**Command:**

```
pwd
```

4. Navigate Deeper Using an Absolute Path

- Move directly into `SubDir1` using an absolute path.

**Command:**

```
cd /home/<your-username>/LabExercise/SubDir1
```

5. Verify Your Position

- Confirm that you are in `SubDir1`.

**Command:**

```
pwd
```

## SECR2043 - OPERATING SYSTEMS (Lab Exercise)

Written by: Dr. Farkhana Muchtar

### 6. Go Back One Level Using Absolute Path

- Return to the LabExercise directory.

**Command:**

```
cd /home/<your-username>/LabExercise
```

### 7. Verify Your Position

- Check you are now back in LabExercise.

**Command:**

```
pwd
```

## 2.3 Exercise 3: Using Relative Paths

A **relative path** depends on your current location. It does not start with / and is “relative” to your current directory.

### 1. Return to Your Home Directory

**Command:**

```
cd
```

### 2. Navigate to SubDir1 Using a Relative Path

- First, move into LabExercise using relative paths, then go into SubDir1. If you are in your home directory, LabExercise should be directly accessible as ~/LabExercise, and then SubDir1 inside it.

**Commands:**

```
cd LabExercise  
cd SubDir1
```

*(Alternatively, if you prefer a single step and LabExercise is in your home directory, you could do `cd ~/LabExercise/SubDir1`, but that uses a tilde, not a purely relative path.)*

### 3. Check Current Directory

- Verify that you are now in SubDir1.

**Command:**

```
pwd
```

## SECR2043 - OPERATING SYSTEMS (Lab Exercise)

Written by: Dr. Farkhana Muchtar

4. Navigate into SubDir2 Using a Relative Path
  - From SubDir1, SubDir2 is right inside it.

**Command:**

```
cd SubDir2
```

5. Check Current Directory
  - Ensure you are now in SubDir2.

**Command:**

```
pwd
```

6. Go Up One Directory (to SubDir1)
  - Use .. to move one level up in the directory tree.

**Command:**

```
cd ..
```

7. Check Current Directory
  - Confirm that you are back in SubDir1.

**Command:**

```
pwd
```

8. Go Up Another Level (to LabExercise)
  - Use .. again to move up from SubDir1 to LabExercise.

**Command:**

```
cd ..
```

9. Check Current Directory
  - Verify that you are in LabExercise.

**Command:**

```
pwd
```

# SECR2043 - OPERATING SYSTEMS (Lab Exercise)

Written by: Dr. Farkhana Muchtar

## 2.4 Exercise 4: Shortcuts and Special Paths

There are several shortcuts you can use to navigate more efficiently:

- `.` (dot) refers to the current directory.
- `..` (double dot) refers to the parent directory (one level up).
- `~` (tilde) refers to your home directory.
- `/` (forward slash) at the beginning refers to the root directory of the entire file system.

### 1. Navigate Using a Combined Relative Path

- From LabExercise, go directly to SubDir2 by specifying both directories: SubDir1/SubDir2.

**Command:**

```
cd SubDir1/SubDir2
```

### 2. Check Your Location

- Verify you are now in SubDir2.

**Command:**

```
pwd
```

### 3. Go Back to LabExercise Using `..`

- Move two levels up using `..` twice to return to LabExercise. Since you are in SubDir2, one `..` takes you to SubDir1, and another `..` takes you to LabExercise.

**Commands:**

```
cd ..  
cd ..
```

*(Or, in one command: `cd ../../`)*

### 4. Check Your Location

- Confirm you are in LabExercise.

**Command:**

```
pwd
```

## SECR2043 - OPERATING SYSTEMS (Lab Exercise)

Written by: Dr. Farkhana Muchtar

### 5. Go Back to Your Home Directory Using Two .. Steps

- If LabExercise is directly inside your home directory, two .. commands from LabExercise would take you back home.

**Commands:**

```
cd ..  
cd ..
```

*(Adjust this based on your actual directory structure. If LabExercise is directly in ~, then just one cd .. is needed. If not, navigate accordingly.)*

### 6. Verify Home Directory

- Check that you are indeed in your home directory.

**Command:**

```
pwd
```

### 7. Navigate to the Root Directory

- Use cd / to go to the root directory of the entire filesystem.

**Command:**

```
cd /
```

### 8. Verify Root Directory

- Check that you are at the root.

**Command:**

```
pwd
```

*(This should display /.)*

### 9. Return Home Using the Tilde ~

- Quickly go back to your home directory using cd ~ or simply cd.

**Command:**

```
cd ~
```

*(or just cd)*

### 10. Final Verification

- Confirm that you are home again.

**Command:**

```
pwd
```

# SECR2043 - OPERATING SYSTEMS (Lab Exercise)

Written by: Dr. Farkhana Muchtar

## 2.5 Exercise 2: File Permission

File permissions in Linux determine who can read, write, and execute files. There are three types of permissions: **read (r)**, **write (w)**, and **execute (x)**, which can be assigned to three categories of users: **owner (u)**, **group (g)**, and **others (o)**. Here is a breakdown of the permission notations:

### 1. Symbolic Notation:

- **u** represents the owner of the file.
- **g** represents the group that the file belongs to.
- **o** represents other users who are not the owner or part of the group.
- **+** adds the specified permission.
- **-** removes the specified permission.
- **r** represents read permission.
- **w** represents write permission.
- **x** represents execute permission.

### 2. Octal Notation:

Each permission is represented by a digit in the octal system (base-8), where read is 4, write is 2, and execute is 1.

The three digits represent permissions for the owner, group, and others, respectively. For example, 644 represents read-write for the owner and read-only for the group and others.

By using the `chmod` command, you can modify file permissions either by specifying the permissions in symbolic notation or by using octal values.

Below are the summary of file permission notation (both symbolic and octal notation).

| Permission        | Symbolic Notation | Octal Notation |
|-------------------|-------------------|----------------|
| Read              | r                 | 4              |
| Write             | w                 | 2              |
| Execute           | x                 | 1              |
| Owner             | u                 | -              |
| Group             | g                 | -              |
| Others            | o                 | -              |
| Add Permission    | +                 | -              |
| Remove Permission | -                 | -              |

In the Octal Notation column, the numbers 4, 2, and 1 represent the read, write, and execute permissions, respectively. You can use these numbers to form different combinations to represent the desired permissions for the owner, group, and others.

# SECR2043 - OPERATING SYSTEMS (Lab Exercise)

Written by: Dr. Farkhana Muchtar

Before we start to use `chmod` command to change file permission,

- i. create dummy files, `file.txt`, `file1.txt`, `file2.txt`, `script.sh` and `reference_file.txt`.

```
touch file.txt file1.txt file2.txt script.sh reference_file.txt
```

## 1. Changing Permissions with Symbolic Notation:

- a. Adds executable permission for the owner of the file (u is for user/owner).

```
ls -l file.txt
chmod u+x file.txt
ls -l file.txt
```

- b. Removes write permission for the group and others (g is for group, o is for others).

```
ls -l file.txt
chmod go-w file.txt
ls -l file.txt
```

- c. Adds read permission for all (owner, group, and others) (a is for all).

```
ls -l file.txt
chmod a+r file.txt
ls -l file.txt
```

## 2. Changing Permissions with Octal Notation:

- a. Sets read, write, and execute permissions for the owner, and read and execute permissions for the group and others.

(Owner:  $7 = 4 + 2 + 1 = \text{Read} + \text{Write} + \text{Execute}$ , Group:  $5 = 4 + 1 = \text{Read, Write}$ , Others:  $5 = 4 + 1 = \text{Read, Write}$ ).

```
ls -l file.txt
chmod 755 file.txt
ls -l file.txt
```

- b. Sets read and write permissions for the owner, and read-only permissions for the group, while removing all permissions for others.

(Owner:  $6 = 4 + 2$ , Group:  $4 = 4$ , Others:  $0$ ).

```
ls -l file.txt
chmod 640 file.txt
ls -l file.txt
```

## SECR2043 - OPERATING SYSTEMS (Lab Exercise)

Written by: Dr. Farkhana Muchtar

### 3. Changing Permissions with Reference File:

- a. Sets the permissions of `file.txt` to match those of `reference_file.txt`.

```
ls -l reference_file.txt file.txt
chmod --reference=reference_file.txt file.txt
ls -l reference_file.txt file.txt
```

### 4. Changing Permissions for Multiple Files:

- a. Sets read and write permissions for the owner, read permissions for the group, and removes all permissions for others for both `file1.txt` and `file2.txt`.

```
ls -l file1.txt file2.txt
chmod u=rw,g=r,o= file1.txt file2.txt
ls -l file1.txt file2.txt
```

- b. Sets read, write, and execute permissions for all (owner, group, and others) for all files with the `.txt` extension in the current directory.

```
ls -l *.txt
chmod 777 *.txt
ls -l *.txt
```

### 5. Changing Permissions Recursively for Directories:

- a. Adds write permission for the owner and all files and directories within directory recursively.

```
ls -lR LabExercise
chmod -R u+w LabExercise
ls -lR LabExercise
```

### 6. Changing Permissions with Special Modes:

- a. Adds executable permission for all (owner, group, and others) using the `+` sign.

```
ls -l script.sh
chmod +x script.sh
ls -l script.sh
```

- b. Sets the group ID on execution, ensuring new files and directories created within directory inherit the group ownership.

```
ls -l LabExercise
chmod g+s LabExercise
ls -l LabExercise
```

## SECR2043 - OPERATING SYSTEMS (Lab Exercise)

Written by: Dr. Farkhana Muchtar

### 2.6 Exercise 3: Counting Number of Files

1. To count files in the current directory, excluding files in subdirectories and hidden files, type the following command:

```
ls | wc -l
```

This command excludes hidden files and does not count files in subdirectories. But it does **not** exclusively count files; it counts all non-hidden entries (both files and directories). If strictly counting only files is required, you'd need a different approach, such as:

```
find . -maxdepth 1 -type f | wc -l
```

2. To count all files in the current directory and its subdirectories, including hidden files, use this command:

```
find . -type f | wc -l
```

3. To count files in the current directory and its subdirectories, but excluding hidden files, run this command:

```
find . -type f -not -path '*/.*' | wc -l
```

This ensures hidden files and directories (those beginning with a dot) are excluded, but regular files are counted.

# SECR2043 - OPERATING SYSTEMS (Lab Exercise)

Written by: Dr. Farkhana Muchtar

## 3 Introduction to Bash Scripting

### 3.1 Exercise 1: Creating a Bash Script

1. Create a new file named `hello.sh` using the `touch` command. Open the file in a text editor (e.g., `nano` or `vi`)

```
nano hello.sh
```

and add the following lines to the top of the file:

```
#!/bin/bash
echo "Hello, World!"
```

2. Save and close the file. This is your first Bash script!
3. To make this script executable (runnable), give execute permissions to `hello.sh` with the command.

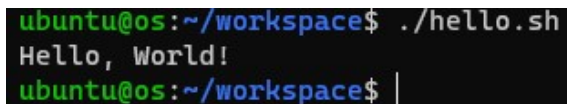
```
chmod u+x hello.sh
```

Run command `ls -l` before and after this command to see the changes.

4. Then, run the script by typing

```
./hello.sh
```

5. You should see "Hello, World!" printed to the console.



```
ubuntu@os:~/workspace$ ./hello.sh
Hello, World!
ubuntu@os:~/workspace$
```

### 3.2 Exercise 2: Using Variables

1. Create a new file named `hello2.sh` using the `touch` command. Open the file in a text editor (e.g., `nano` or `vi`)

```
nano hello2.sh
```

and add the following lines to the top of the file:

```
#!/bin/bash
GREETING="Hello, World!"
echo $GREETING
```

2. Save and close the file.
3. Give execute permissions to `hello2.sh` with the command

```
chmod u+x hello2.sh
```

4. Then, run the script by typing.
- ```
./hello2.sh
```

## SECR2043 - OPERATING SYSTEMS (Lab Exercise)

Written by: Dr. Farkhana Muchtar

5. You should see "Hello, World!" printed to the console.

```
ubuntu@os:~/workspace$ nano hello2.sh
ubuntu@os:~/workspace$ chmod u+x hello2.sh
ubuntu@os:~/workspace$ ./hello2.sh
Hello, World!
ubuntu@os:~/workspace$ |
```

### 3.3 Exercise 3: Using Variables for Arithmetic

1. Create a new file named `variable.sh` using the `touch` command. Open the file in a text editor (e.g., `nano` or `vi`)

```
nano variable.sh
```

and add the following lines to the top of the file:

```
#!/bin/bash

NUMBER1=10
NUMBER2=5
SUM=$((NUMBER1 + NUMBER2))
echo "The sum of $NUMBER1 and $NUMBER2 is: $SUM"
```

2. Save and close the file.
3. Give execute permissions to `variable.sh` with the command

```
chmod u+x variable.sh
```

4. Then, run the script by typing.

```
./variable.sh
```

5. You should see sum of the two numbers, printed to the console.

## SECR2043 - OPERATING SYSTEMS (Lab Exercise)

Written by: Dr. Farkhana Muchtar

### 3.4 Exercise 4: Getting User Input

1. Create a new file named `hello3.sh` using the `touch` command. Open the file in a text editor (e.g., `nano` or `vi`)

```
nano hello3.sh
```

and add the following lines to the top of the file:

```
#!/bin/bash
echo "What's your name?"
read NAME
echo "Hello, $NAME!"
```

2. Save and close the file.
3. Give execute permissions to `hello3.sh` with the command

```
chmod u+x hello3.sh
```

4. Then, run the script by typing.

```
./hello3.sh
```

5. The program will asking your name and displaying hello message, following with your name, printed to the console

### 3.5 Exercise 5: Getting User Input for Arithmetic

1. Create a new file named `input.sh` using the `touch` command. Open the file in a text editor (e.g., `nano` or `vi`)

```
nano input.sh
```

and add the following lines to the top of the file:

```
#!/bin/bash
echo "Enter the first number:"
read NUMBER1

echo "Enter the second number:"
read NUMBER2

SUM=$((NUMBER1 + NUMBER2))
PRODUCT=$((NUMBER1 * NUMBER2))

echo "The sum of $NUMBER1 and $NUMBER2 is: $SUM"
echo "The product of $NUMBER1 and $NUMBER2 is: $PRODUCT"
```

## SECR2043 - OPERATING SYSTEMS (Lab Exercise)

Written by: Dr. Farkhana Muchtar

2. Save and close the file.
3. Give execute permissions to `input.sh` with the command  

```
chmod u+x input.sh
```
4. Then, run the script by typing.  

```
./input.sh
```
5. You should see sum and product of two numbers, printed to the console

### 3.6 Exercise 6: Conditional Statements

1. Create a new file named `if_statement.sh` using the `touch` command. Open the file in a text editor (e.g., `nano` or `vi`)

```
nano if_statement.sh
```

and add the following lines to the top of the file:

```
#!/bin/bash
echo "Enter the first number:"
read NUMBER1

echo "Enter the second number:"
read NUMBER2

SUM=$((NUMBER1 + NUMBER2))
PRODUCT=$((NUMBER1 * NUMBER2))

echo "The sum of $NUMBER1 and $NUMBER2 is: $SUM"
echo "The product of $NUMBER1 and $NUMBER2 is: $PRODUCT"

if [ $SUM -gt $PRODUCT ]
then
    echo "The sum is greater than the product."
elif [ $SUM -lt $PRODUCT ]
then
    echo "The product is greater than the sum."
else
    echo "The sum and product are equal."
fi
```

2. Save and close the file.
3. Give execute permissions to `if_statement.sh` with the command  

```
chmod u+x if_statement.sh
```
4. Then, run the script by typing.  

```
./if_statement.sh
```
5. You should see either the status is sum is bigger than product or product is bigger than sum, printed to the console

## SECR2043 - OPERATING SYSTEMS (Lab Exercise)

Written by: Dr. Farkhana Muchtar

### 3.7 Exercise 7: Loops

1. Create a new file named `loop.sh` using the touch command. Open the file in a text editor (e.g., nano or vi)

```
nano loop.sh
```

and add the following lines to the top of the file:

```
#!/bin/bash
echo "Counting from 1 to 10:"

for ((i=1; i<=10; i++))
do
    SQUARE=$((i * i))
    echo "The square of $i is: $SQUARE"
done
```

2. Save and close the file.
3. Give execute permissions to `loop.sh` with the command

```
chmod u+x loop.sh
```

4. Then, run the script by typing.

```
./loop.sh
```

5. You should see list of number from 1-10 and square of the numbers, printed to the console.

## SECR2043 - OPERATING SYSTEMS (Lab Exercise)

Written by: Dr. Farkhana Muchtar

### 3.8 Exercise 8: Running Several Linux Commands

1. Create a new file named `file_management.sh` using the `touch` command. Open the file in a text editor (e.g., `nano` or `vi`)

```
nano file_management.sh
```

and add the following lines to the top of the file:

```
#!/bin/bash

# Create a new directory and navigate into it
mkdir Exercise; cd Exercise

# Create subdirectories: SubDir1 and SubDir1/SubSubDir
mkdir SubDir1; mkdir SubDir1/SubSubDir

# Navigate to the SubSubDir directory
cd SubDir1/SubSubDir

# Echo a message into a text file
echo "This is some text." > file.txt

# Display the content of the text file
cat file.txt

# Navigate back to the Exercise directory
cd ../../

# List the contents of the Exercise directory
ls

# Display the current working directory
pwd
```

2. Save and close the file.
3. Give execute permissions to `file_management.sh` with the command

```
chmod u+x file_management.sh
```

4. Then, run the script by typing.

```
./file_management.sh
```

5. You should see file management in action, printed to the console.

## SECR2043 - OPERATING SYSTEMS (Lab Exercise)

Written by: Dr. Farkhana Muchtar

### 3.9 Exercise 9: Modify File Permission

1. Create a new file named `file_permission.sh` using the `touch` command. Open the file in a text editor (e.g., `nano` or `vi`)

```
nano file_permission.sh
```

and add the following lines to the top of the file:

```
#!/bin/bash
echo "Enter the filename:"
read FILENAME

echo "Enter the desired permissions (in numeric format):"
read PERMISSIONS

chmod $PERMISSIONS $FILENAME

echo "Permissions for $FILENAME have been set to $PERMISSIONS."
```

2. Save and close the file.
3. Give execute permissions to `file_permission.sh` with the command

```
chmod u+x file_permission.sh
```

4. Then, run the script by typing.

```
./file_permission.sh
```

5. You should see file permission status, printed to the console.

### 3.10 Exercise 10: Modify Multiple File Permission Based On Extension File Using For Loop

11. **Create multiple .sh files:**  
For demonstration, create a few shell script files:

```
touch script1.sh script2.sh script3.sh
```

Add simple content to each file (optional):

```
echo '#!/usr/bin/env bash' > script1.sh
echo 'echo "Hello from script1.sh"' >> script1.sh

echo '#!/usr/bin/env bash' > script2.sh
echo 'echo "Hello from script2.sh"' >> script2.sh

echo '#!/usr/bin/env bash' > script3.sh
echo 'echo "Hello from script3.sh"' >> script3.sh
```

## SECR2043 - OPERATING SYSTEMS (Lab Exercise)

Written by: Dr. Farkhana Muchtar

12. **Verify current permissions** (not necessarily executable yet):

```
ls -l *.sh
```

You might see something like:

```
-rw-r--r-- 1 user user 42 Dec 12 10:00 script1.sh
-rw-r--r-- 1 user user 42 Dec 12 10:00 script2.sh
-rw-r--r-- 1 user user 42 Dec 12 10:00 script3.sh
```

13. **Write the Bash script that adjusts permissions:**

Create a file named `set_executable.sh`:

```
nano set_executable.sh
```

Add the following code:

```
#!/usr/bin/bash
# set_executable.sh
# This script uses a for loop to update all *.sh files to be executable.
```

```
for file in *.sh
do
    # Check if the file exists and is not a directory
    if [ -f "$file" ]; then
        chmod +x "$file"
        echo "Set $file to executable."
    fi
done
```

14. **Make the script itself executable:**

```
chmod +x set_executable.sh
```

15. **Run the script:**

```
./set_executable.sh
```

Output should look like:

```
Set script1.sh to executable.
Set script2.sh to executable.
Set script3.sh to executable.
```

16. **Verify permissions again:**

```
ls -l *.sh
```

Now the files should have executable permissions (e.g., `-rwxr-xr-x`).